ynov CAMPUS

Coder avec l'IA : GitHub Copilot, Cursor et Claude

Outils IA · Bachelor 1 · Séance 2 sur 3 · 4 h

Ce que vous saurez faire à la fin de la séance

Configurer un assistant

Copilot dans VS Code ou Cursor, prêt à l'emploi.

Utiliser les bons modes

Autocomplétion, chat, édition ciblée, mode agent.

Donner le bon contexte

Fichiers, références, consignes de projet.

Comprendre et déboguer

Expliquer du code, corriger une erreur, écrire des tests simples.

Où en est TaskFlow ?

Séance 1 ✓
Cahier des charges, user stories, dépôt GitHub

→

Séance 2 · aujourd'hui
Développer l'application avec l'IA

→

Séance 3
Audit, corrections et oral



Objectif du jour : repartir avec une application qui fonctionne (créer, afficher, modifier, supprimer des tâches, sauvegarde) et votre fonctionnalité filière commencée.

Déroulé de la séance



30 min	45 min	2 h 30	15 min
Coder avec l'IA : principes	Démo Copilot, Cursor, Claude	Exercice TaskFlow en 3 sprints	Synthèse

Livrables de fin de séance
Une application fonctionnelle sur GitHub, un commit par sprint au minimum, et PROMPTS.md à jour avec au moins un prompt par sprint.

1

Coder avec l'IA : les principes

30 min

Trois façons de coder avec l'IA

Compléter

Vous tapez, l'IA propose la suite en gris. Tab pour accepter.

Autonomie de l'IA



Discuter

Vous posez une question sur le code, l'IA explique ou propose un changement.

Autonomie de l'IA



Déléguer

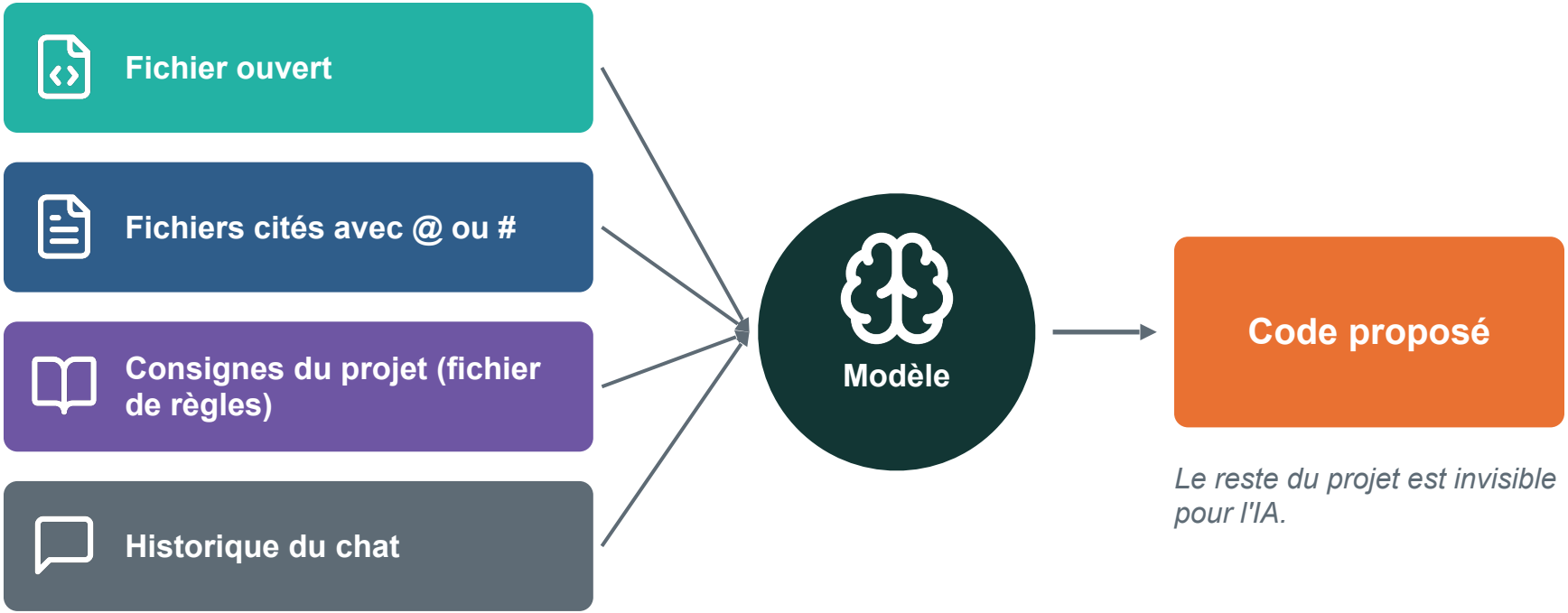
Mode agent : l'IA modifie plusieurs fichiers et lance des commandes.

Autonomie de l'IA



Plus l'IA est autonome, plus vous devez relire

Le contexte : ce que l'IA voit vraiment



Le reste du projet est invisible pour l'IA.

La règle d'or du module



On ne garde aucune ligne de code que l'on n'est pas capable d'expliquer.

Si vous ne comprenez pas : demandez à l'IA de l'expliquer, simplifiez, ou réécrivez.

Copilot, Cursor, Claude : lequel pour quoi ?

	GitHub Copilot	Cursor	Claude (chat)
Où	Extension de VS Code	Éditeur complet (basé sur VS Code)	Navigateur ou application
Points forts	Autocomplétion, intégré à GitHub	Édition multi-fichiers, mode agent	Explications, conception, débogage
Accès étudiant	Gratuit avec le Student Developer Pack	Version gratuite limitée	Version gratuite limitée
Idéal pour	Écrire du code au fil de l'eau	Modifier tout le projet	Comprendre avant de coder

2

Démonstration en direct

45 min

Installation : la check-list

GitHub Copilot

1. Demander le GitHub Student Developer Pack
2. Installer l'extension GitHub Copilot dans VS Code
3. Se connecter avec son compte GitHub

Cursor

1. Télécharger Cursor sur cursor.com
2. Importer ses réglages VS Code si besoin
3. Ouvrir le dossier TaskFlow

Test rapide

Écrire un commentaire « // fonction qui additionne deux nombres » et vérifier qu'une suggestion apparaît.

Du commentaire au code

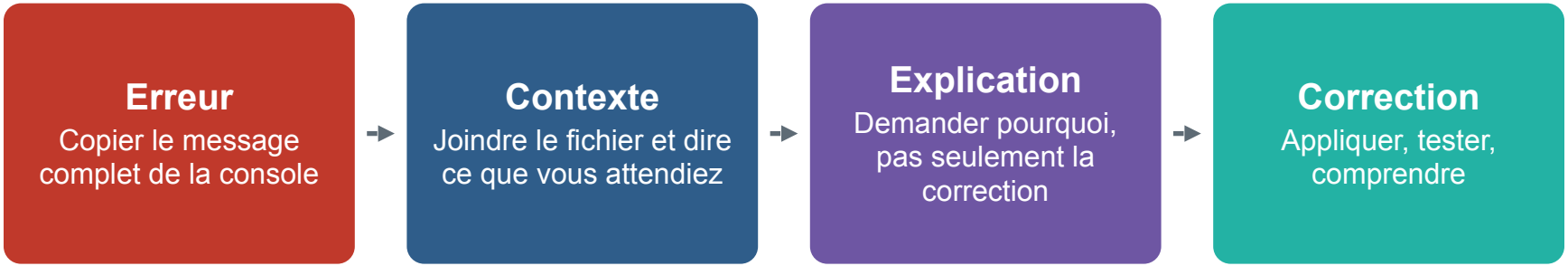
app.js

```
// Ajoute une tâche à la liste à partir du
formulaire.
// Refuse un titre vide. Priorité par défaut :
"moyenne".
function addTask(title, priority = "moyenne") {
  if (!title.trim()) return null;
  const task = {
    id: Date.now(),
    titre: title.trim(),
    priorite: priority,
    statut: "a_faire",
  };
  tasks.push(task);
  render();
  return task;
}
```

À observer

- **Commentaire précis** = code précis
- **render()** existe-t-il vraiment ?
- **Date.now()** comme id : suffisant ici ?
- **Cas limites :** titre avec seulement des espaces

Utiliser le chat pour déboguer




```
Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')
    at app.js:12
```

Prompt : « Voici l'erreur et mon app.js. Explique la cause en 3 phrases, puis propose la correction la plus simple. »

Les bons réflexes de prompt pour le code

Une tâche à la fois

« Ajoute la suppression d'une tâche », pas « fais toute l'app ».

Fixez les contraintes

« JavaScript sans framework, pas de bibliothèque externe ».

Faites expliquer

« Explique chaque ligne comme à un débutant ».

Demandez des tests

« Donne 3 cas de test, dont un cas limite ».

Même demande dans trois outils



GitHub Copilot

Code produit ?
Fichiers modifiés ?
Contexte compris ?



Cursor

Code produit ?
Fichiers modifiés ?
Contexte compris ?



Claude

Code produit ?
Fichiers modifiés ?
Contexte compris ?

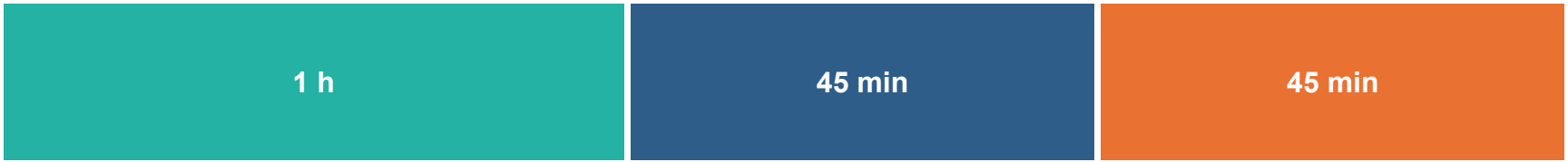
Demande : « Ajoute un filtre pour afficher seulement les tâches terminées ou à faire. »

3

Exercice : développer TaskFlow

3 sprints, 2 h 30

Le plan : trois sprints



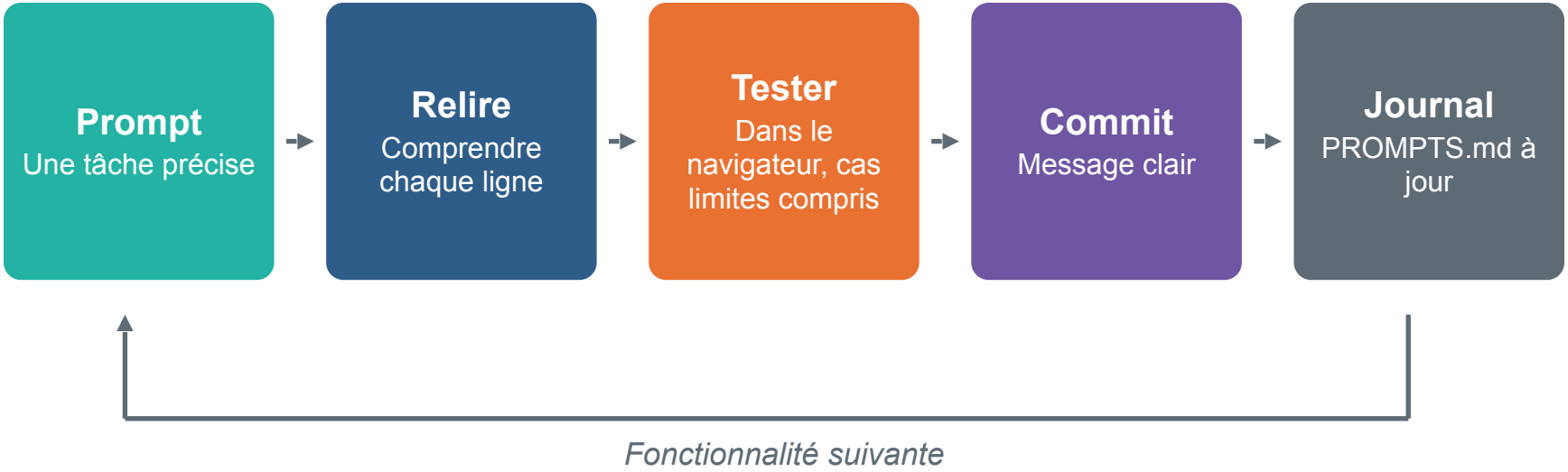
Sprint 1 · Fonctions de base : créer, afficher, modifier, supprimer, terminer

Sprint 2 · Sauvegarde (localStorage) et mise en forme CSS

Sprint 3 · Fonctionnalité filière

À la fin de chaque sprint : un commit sur GitHub et une entrée dans PROMPTS.md (outil, code gardé, modifié ou rejeté).

Le cycle à suivre pour chaque fonctionnalité



Sprint 2 : sauvegarder avec localStorage

app.js

```
function save() {
  localStorage.setItem("tasks",
JSON.stringify(tasks));
}

function load() {
  const data = localStorage.getItem("tasks");
  tasks = data ? JSON.parse(data) : [];
}
```

Questions à vous poser

- Pourquoi JSON.stringify ?
- Que se passe-t-il au premier lancement ?
- Où appeler save() ?
- Les données sont-elles privées ?

localStorage est visible par n'importe qui ayant accès au navigateur : on y reviendra en séance 3.

Sprint 3 : votre fonctionnalité filière

Développement

Trier par priorité ou échéance, filtrer par statut, ajouter des catégories.

Cybersécurité

Écran de connexion et contrôle de robustesse du mot de passe (longueur, majuscules, chiffres, caractères spéciaux).

Synthèse de la séance

Où l'IA a aidé

Code répétitif, syntaxe, explications d'erreurs.

Où elle a fait perdre du temps

Code inventé, mauvais contexte, corrections en boucle.

Séance 3 : audit du code d'un autre binôme et oral de 10 min par groupe. Tirage au sort de l'ordre de passage maintenant.